CS 639 Homework 1: Values and Value Iteration

- **Due** Friday, October 2, on Gradescope (the exact time is there); hard close 48 hours later. Slip days (four shared by HW1–HW4, at most two here) are applied automatically; late time beyond them costs 10% of the maximum score per started 24-hour period.
- **Weight.** 10% of the course grade; 100 points: Problems 1–4 carry 24, 16, 24, 36. Problems 3–4 are the programming part, 60 points, graded from the printed output, the tables, and the figure in your report. Most students should expect about three hours. The Assistance Statement at the end is required.
- **What to submit.** One file: `hw1.pdf`, typeset in LaTeX from the template `hw1.tex` (handwritten or scanned work is not accepted). Problems 1 and 2 are solved by hand: under each part, write the equations and the computation that lead to your answer, then the answer; the template marks the place, and its `solution` environment typesets your work in blue. Problem 3 is the complete printed output of `hw1.py`; Problem 4 is three tables filled in from that output, and the figure the script saves. `hw1.py` itself is not submitted. No part asks for an explanation in words.
- **Grading.** In Problems 1 and 2 the derivation carries the credit: a correct method with an arithmetic slip loses little, a bare number without its derivation earns at most half, and a part that reuses a slipped number from an earlier part is graded on its own logic. In Problems 3 and 4 the printed output and the tables are graded as they stand.
- **Collaboration and AI.** Discuss ideas with at most two named classmates; write every derivation, every line of code, and every answer yourself. Problem 1(b) is an individual derivation check (**Red**: no AI). Everything else is **Yellow**: limited, disclosed use, never for the computations or the code you are asked to produce. List every collaborator, resource, and tool in the Assistance Statement.
- **What you need.** Lectures 2–5 and Sutton and Barto, Chapter 3 and Sections 4.1 and 4.4. Each problem states the definitions and results it uses. Start with Problem 1 and the code; ask on Piazza early.

Notation. An MDP is $(\mathcal{S}, \mathcal{A}, P, r, \gamma, d_0)$ with finite $\mathcal{S}$ and $\mathcal{A}$, $r(s,a) = \mathbb{E}[R_{t+1} \mid S_t = s, A_t = a]$, and $\gamma \in [0,1)$ unless said otherwise; terminal states are absorbing with reward 0; $R_{\max}$ bounds $|R_{t+1}|$. A deterministic policy π leaves the chain $P^\pi(s' \mid s) = P(s' \mid s, \pi(s))$, $r^\pi(s) = r(s, \pi(s))$. V^π, Q^π are the values of π, $V^\star, Q^\star$ the optimal values, $\pi^\star$ an optimal policy, and $J(\pi) = \mathbb{E}_{S_0 \sim d_0}[V^\pi(S_0)]$. For $V \in \mathbb{R}^{|\mathcal{S}|}$ the lookahead table and the two Bellman operators are

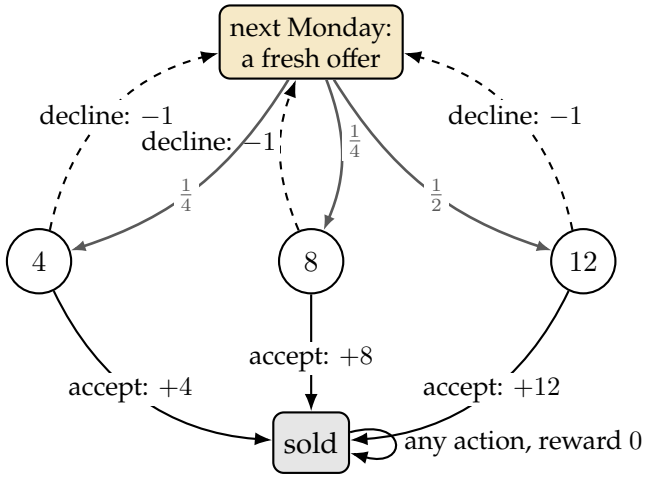
$$Q_V(s,a) \coloneqq r(s,a) + \gamma \sum_{s'} P(s' \mid s, a)\, V(s'),$$

$$(\mathcal{T}^\pi V)(s) \coloneqq Q_V\big(s, \pi(s)\big) = r^\pi(s) + \gamma \sum_{s'} P^\pi(s' \mid s)\, V(s'), \qquad (\mathcal{T} V)(s) \coloneqq \max_a Q_V(s,a),$$

so $V^\pi = \mathcal{T}^\pi V^\pi$, $V^\star = \mathcal{T}V^\star$, $Q_{V^\pi} = Q^\pi$, $Q_{V^\star} = Q^\star$. A deterministic π is *greedy with respect to* V if $\pi(s) \in \arg\max_a Q_V(s,a)$ for every s; then $\mathcal{T}^\pi V = \mathcal{T}V$. The norm is $\|V\|_\infty = \max_s |V(s)|$; inequalities between vectors hold entry by entry. Value iteration is $V_{k+1} = \mathcal{T}V_k$ from $V_0 = 0$ unless said otherwise, each V_{k+1} computed from the whole of V_k (Lecture 5's box, not in place); π_k is the greedy policy of the k-th iterate.

1 Selling a Bike: The Optimality Equation by Hand [24 points]

You are selling a used bike online. Every Monday exactly one buyer arrives with a take-it-or-leave-it offer of 4, 8, or 12 (hundreds of dollars) with probabilities $\frac{1}{4}, \frac{1}{4}, \frac{1}{2}$, independently of the past. Accepting ends the sale; declining sends the buyer away for good, costs 1 to keep the listing up, and brings a fresh buyer next Monday. The discount is $\gamma = 0.9$ per week. As an MDP: the state is the offer on the table, $w \in \{4, 8, 12\}$, plus the terminal state *sold*; the actions are accept and decline; accepting moves to sold with reward w, declining moves to a fresh offer, drawn with the probabilities above, with reward -1; $d_0 = (\frac{1}{4}, \frac{1}{4}, \frac{1}{2})$, the first Monday's offer.



(a) **[8 points]** Write the Bellman optimality equation $V^\star = \mathcal{T}V^\star$ at the three offers. Show that it takes the form $V^\star(w) = \max\{w, R\}$ with one number $R = Q^\star(w, \text{decline})$, the same for all three offers, and that R solves one equation in R alone. Solve that equation: which offers exceed R fixes the form of the equation, so treat the cases (at most four) and keep those whose solution is consistent with the case. Give R, the three values $V^\star(4), V^\star(8), V^\star(12)$, and *every* optimal deterministic policy.

Solution.

Three values are:

$$V^*(4) = 8 \quad V^*(8) = 8 \quad V^*(12) = 12$$

Best policies are:

$$(D, D, A) \quad (D, A, A)$$

$$V^*(4) = max\{4, -1 + 0.9 * (\frac{1}{4}V^*(4) + \frac{1}{4}V^*(8) + \frac{1}{2}V^*(12))\}$$
$$V^*(8) = max\{8, -1 + 0.9 * (\frac{1}{4}V^*(4) + \frac{1}{4}V^*(8) + \frac{1}{2}V^*(12))\}$$
$$V^*(12) = max\{12, -1 + 0.9 * (\frac{1}{4}V^*(4) + \frac{1}{4}V^*(8) + \frac{1}{2}V^*(12))\}$$

We know,

$$-1 + 0.9 * (\frac{1}{4}V^*(4) + \frac{1}{4}V^*(8) + \frac{1}{2}V^*(12)) < \frac{1}{4}V^*(4)) + \frac{1}{4}V^*(8) + \frac{1}{2}V^*(12) \leq V^*(12)$$

Thus,

$$V^*(12) = 12$$

If $V^*(8) > 8$, then we have,

$$V^*(8) = V^*(4) = -1 + 0.9 * (\frac{1}{4}V^*(4) + \frac{1}{4}V^*(8) + \frac{1}{2}V^*(12)) \Rightarrow V^*(8) = 8$$

Contradict! Therefore,

$$V^*(8) = 8$$

Then we can easily know

$$V^*(4) = 8$$

There are two optimal policies:

state	action
4	decline
8	decline
12	accept

state	action
4	decline
8	accept
12	accept

(b) **[10 points]** **[Red]** Run value iteration from $V_0 = 0$, each V_{k+1} computed from the whole of V_k ($V_k(\text{sold}) = 0$ throughout). Write $R_k := Q_{V_k}(w, \text{decline}) = -1 + 0.9\big(\frac{1}{4}V_k(4) + \frac{1}{4}V_k(8) + \frac{1}{2}V_k(12)\big)$ for the decline entry of the lookahead table of V_k, so that $V_{k+1}(w) = \max\{w, R_k\}$. Compute V_1, V_2, V_3 and the greedy policies $\pi_0, \ldots, \pi_3$. Let $e_k := \|V_k - V^\star\|_\infty$ with $V^\star$ from (a). Show that $e_{k+1} = \frac{\gamma}{4}\, e_k$ for every $k \geq 1$ (you may use that $7.1 \leq R_k < 8$ for all $k \geq 1$), find the first k with $e_k \leq 0.01$, and give the number of iterations Lecture 5's worst-case bound guarantees for accuracy $\varepsilon = 0.01$, $\lceil \log\big(R_{\max}/((1-\gamma)\varepsilon)\big)/\log(1/\gamma) \rceil$ (here $R_{\max} = 12$).

Solution.

Because

$$V_{k+1}(s) = max\{s, -1 + 0.9 * (\frac{1}{4}V_k(4) + \frac{1}{4}V_k(8) + \frac{1}{2}V_k(12))\}$$

and

$$V_0(4) = 0 \quad V_0(8) = 0 \quad V_0(12) = 0$$

The first iteration is as follows:

$$\pi_0 = (Accept, Accept, Accept)$$

$$V_1(4) = max\{4, -1 + 0.9 * (\frac{1}{4} * 0 + \frac{1}{4} * 0 + \frac{1}{2} * 0)\} = 4$$
$$V_1(8) = max\{8, -1 + 0.9 * (\frac{1}{4} * 0 + \frac{1}{4} * 0 + \frac{1}{2} * 0)\} = 8$$
$$V_1(12) = max\{12, -1 + 0.9 * (\frac{1}{4} * 0 + \frac{1}{4} * 0 + \frac{1}{2} * 0)\} = 12$$

The second iteration is as follows:

$$\pi_1 = (Decline, Accept, Accept)$$

$$V_2(4) = max\{4, -1 + 0.9 * (\frac{1}{4} * 4 + \frac{1}{4} * 8 + \frac{1}{2} * 12)\} = 7.1$$
$$V_2(8) = max\{8, -1 + 0.9 * (\frac{1}{4} * 4 + \frac{1}{4} * 8 + \frac{1}{2} * 12)\} = 8$$
$$V_2(12) = max\{12, -1 + 0.9 * (\frac{1}{4} * 4 + \frac{1}{4} * 8 + \frac{1}{2} * 12)\} = 12$$

The third iteration is as follows:

$$\pi_2 = (Decline, Accept, Accept)$$

$$V_3(4) = max\{4, -1 + 0.9 * (\frac{1}{4} * 7.1 + \frac{1}{4} * 8 + \frac{1}{2} * 12)\} = 7.7975$$
$$V_3(8) = max\{8, -1 + 0.9 * (\frac{1}{4} * 7.1 + \frac{1}{4} * 8 + \frac{1}{2} * 12)\} = 8$$
$$V_3(12) = max\{12, -1 + 0.9 * (\frac{1}{4} * 7.1 + \frac{1}{4} * 8 + \frac{1}{2} * 12)\} = 12$$

Since the $V_3(4) < 8$,

$$\pi_3 = (Decline, Accept, Accept)$$

$$e_{k+1} = \|V_{k+1} - V^\star\|_\infty = |8 - V_{k+1}(4)|$$
$$e_k = \|V_k - V^\star\|_\infty = |8 - V_k(4)|$$

Therefore, we have

$$\begin{aligned} e_{k+1} &= 8 - V_{k+1}(4) \\ &= 8 - (-1 + 0.9 * (\frac{1}{4} * V_k + \frac{1}{4} * 8 + \frac{1}{2} * 12)) \\ &= 8 - (-1 + 0.9 * ((-1) * \frac{1}{4} * (8 - V_k) + 2 * \frac{1}{4} * 8 + \frac{1}{2} * 12)) \\ &= 8 - (-1 + 0.9 * ((-1) * \frac{1}{4} * e_k + 2 * \frac{1}{4} * 8 + \frac{1}{2} * 12)) \\ &= \frac{\gamma}{4} e_k \end{aligned}$$

Because

$$e_1 = 8 - V_1(4) = 4$$

Then we want

$$4 * (\frac{\gamma}{4})^{k-1} \leq 0.01$$

We have the first k is,

$$k = 6$$

The worst guarantee is

$$\lceil \frac{\log\big(R_{\max}/((1-\gamma)\varepsilon)\big)}{\log(1/\gamma)} \rceil = \lceil \frac{\log\big(12/(0.1 * 0.01)\big)}{\log(1/0.9)} \rceil = 90$$

(c) **[6 points]** Add 1 to every reward: accepting pays $w+1$, declining pays 0, "sold" still pays 0. Find the new decline value $R' := Q^\star(w, \text{decline})$ at $\gamma = 0.9$, by the case analysis of (a), and every optimal deterministic policy.

Solution.

$$R' = 0 + 0.9 * (\frac{1}{4}V^*(4) + \frac{1}{4}V^*(8) + \frac{1}{2}V^*(12)) = 0.9 * (\frac{1}{4}V^*(4) + \frac{1}{4}V^*(8) + \frac{1}{2}V^*(12))$$

Thus,

$$V^*(4) = max\{5, R'\}$$
$$V^*(8) = max\{9, R'\}$$
$$V^*(12) = max\{13, R'\}$$

Because we have,

$$R' \leq 0.9 * V^*(12) < V^*(12)$$

then,

$$V^*(12) = 13$$

Suppose $R' \leq 9$, then

$$V^*(8) = 9 \Rightarrow R' > 5 \Rightarrow V^*(4) = R' \Rightarrow R' > 9$$

Contradicts! Thus, $R' > 9$ and we have,

$$V^*(4) = V^*(8) = R' \Rightarrow R' = \frac{117}{11}$$

Therefore, we have

$$V^*(4) = \frac{117}{11} \quad V^*(8) = \frac{117}{11} \quad V^*(12) = 13$$

The unique optimal policy is:

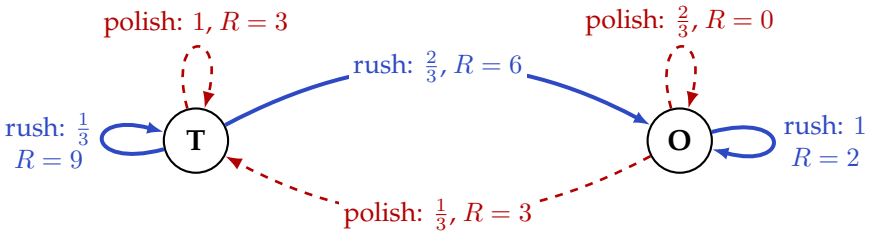
$$(Decline, Decline, Accept)$$

state	action
4	decline
8	decline
12	accept

Where this comes from: Lecture 2 (the MDP, terminal states, the discount, shifting all rewards), Lecture 4 (the optimality equation, greedy extraction, ties), Lecture 5 (value iteration by hand, the rate, the iteration count); Sutton and Barto, Sections 3.1–3.6 and 4.4.

2 A Channel: Policy Evaluation and the Optimality Test [16 points]

A creator posts one video a week. The channel is either *trending* (T) or *obscure* (O), and each week the creator either *rushes* a video out or *polishes* one. Rushing while trending pays 9 if the channel stays trending, which happens with probability $\frac{1}{3}$, and 6 if it drops to obscure; polishing while trending pays 3 and keeps the channel trending. Rushing while obscure pays 2 and keeps it obscure; polishing while obscure pays 3 if the channel starts trending, with probability $\frac{1}{3}$, and 0 otherwise. $\gamma = 0.9$; the channel starts obscure. Write (a_T, a_O) for the deterministic policy that takes a_T when trending and a_O when obscure. (`hw1.py` contains this model as `channel()`; once your code of Problem 3 works, you can check your hand computations with it.)



Solid: rush. Dashed: polish. Each arrow carries the probability of the transition and the reward R_{t+1} paid on it.

(a) **[8 points]** The policy $\pi = (\text{polish}, \text{polish})$. Compute $r(s,a) = \mathbb{E}[R_{t+1} \mid S_t = s, A_t = a]$ for the four pairs and write the chain (P^π, r^π). Solve its two Bellman equations $V^\pi = r^\pi + \gamma P^\pi V^\pi$ by hand, compute $Q^\pi(s,a) = Q_{V^\pi}(s,a)$ for the four pairs, and determine the state(s) at which Lecture 4's test, $V^\pi(s) = \max_a Q^\pi(s,a)$, fails.

Solution.

$$
\begin{aligned}
r(T, Polish) &= 3 \\
r(T, Rush) &= \frac{1}{3} \times 9 + \frac{2}{3} \times 6 = 7 \\
r(O, Polish) &= \frac{1}{3} \times 3 + \frac{2}{3} \times 0 = 1 \\
r(O, Rush) &= 2
\end{aligned}
$$

Since $\pi = (\text{polish}, \text{polish})$, the probability transition matrix P and r^π are:

$$
P^\pi = \begin{pmatrix} 1 & 0 \\ \frac{1}{3} & \frac{2}{3} \end{pmatrix} \quad r^\pi = \begin{pmatrix} 3 \\ 1 \end{pmatrix}
$$

Then we have,

$$
\begin{aligned}
V^\pi &= r^\pi + \gamma P^\pi V^\pi \\
\Rightarrow \begin{pmatrix} v_1 \\ v_2 \end{pmatrix} &= \begin{pmatrix} 3 \\ 1 \end{pmatrix} + \frac{9}{10} \times \begin{pmatrix} 1 & 0 \\ \frac{1}{3} & \frac{2}{3} \end{pmatrix} \begin{pmatrix} v_1 \\ v_2 \end{pmatrix} \\
\Rightarrow \begin{pmatrix} \frac{1}{10} v_1 \\ -\frac{3}{10} v_1 + \frac{2}{5} v_2 \end{pmatrix} &= \begin{pmatrix} 3 \\ 1 \end{pmatrix}
\end{aligned}
$$

Thus,

$$
V^\pi(T) = 30, V^\pi(O) = 25
$$

Then we have,

$$
\begin{aligned}
Q^\pi(T, Polish) &= 3 + \frac{9}{10} \times V^\pi(T) = 30 \\
Q^\pi(T, Rush) &= 7 + \frac{9}{10} \times (\frac{1}{3} V^\pi(T) + \frac{2}{3} V^\pi(O)) = 31 \\
Q^\pi(O, Polish) &= 1 + \frac{9}{10} \times (\frac{1}{3} V^\pi(T) + \frac{2}{3} V^\pi(O)) = 25 \\
Q^\pi(O, Rush) &= 2 + \frac{9}{10} \times V^\pi(O) = 24.5
\end{aligned}
$$

Because

$$\max_a Q^\pi(T,a) = Q^\pi(T,R) = 31 > V^\pi(T)$$
$$\max_a Q^\pi(O,a) = Q^\pi(O,P) = 25 = V^\pi(O)$$

The state O passes and the state T fails. Thus, π is not the best policy.

(b) **[8 points]** Let π' be the policy greedy with respect to Q^π from (a). Write it, solve its two Bellman equations by hand (the two rows of $P^{\pi'}$ are identical), compute $Q^{\pi'}(s,a)$ for the four pairs, and run the test. Conclude $V^\star$ and $J(\pi^\star)$.

Solution. According to (a),

$$\pi' = (Rush, Polish)$$

Then The matrix $P^{\pi'}$ and $r^{\pi'}$ are:

$$P^{\pi'} = \begin{pmatrix} \frac{1}{3} & \frac{2}{3} \\ \frac{1}{3} & \frac{2}{3} \end{pmatrix} \quad r^{\pi'} = \begin{pmatrix} 7 \\ 1 \end{pmatrix}$$

Then we have,

$$V^\pi = r^\pi + \gamma P^\pi V^\pi$$
$$\Rightarrow \begin{pmatrix} v_1 \\ v_2 \end{pmatrix} = \begin{pmatrix} 7 \\ 1 \end{pmatrix} + \frac{9}{10} \times \begin{pmatrix} \frac{1}{3} & \frac{2}{3} \\ \frac{1}{3} & \frac{2}{3} \end{pmatrix} \begin{pmatrix} v_1 \\ v_2 \end{pmatrix}$$
$$\Rightarrow \begin{pmatrix} \frac{7}{10}v_1 - \frac{3}{5}v_2 \\ -\frac{3}{10}v_1 + \frac{2}{5}v_2 \end{pmatrix} = \begin{pmatrix} 7 \\ 1 \end{pmatrix}$$

Thus,

$$V^{\pi'}(T) = 34, \quad V^{\pi'}(O) = 28$$

Then we have,

$$Q^{\pi'}(T, Polish) = 3 + \frac{9}{10} \times V^{\pi'}(T) = 33.6$$
$$Q^{\pi'}(T, Rush) = 7 + \frac{9}{10} \times (\frac{1}{3}V^{\pi'}(T) + \frac{2}{3}V^{\pi'}(O)) = 34$$
$$Q^{\pi'}(O, Polish) = 1 + \frac{9}{10} \times (\frac{1}{3}V^{\pi'}(T) + \frac{2}{3}V^{\pi'}(O)) = 28$$
$$Q^{\pi'}(O, Rush) = 2 + \frac{9}{10} \times V^{\pi'}(O) = 27.2$$

Test is as follows:

$$V^{\pi'}(T) = \max_a Q^{\pi'}(T,a) = 34$$
$$V^{\pi'}(O) = \max_a Q^{\pi'}(O,a) = 28$$

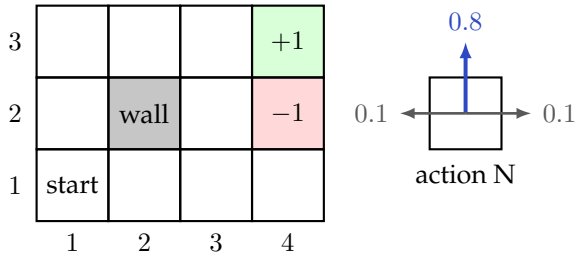
Therefore,

$$J(\pi^*) = V^*(O) = 28 \quad \pi^* = (Rush, Polish) \quad V^* = \begin{pmatrix} 34 \\ 28 \end{pmatrix}$$

Where this comes from: Lecture 3 (the chain of a policy, V^π and Q^π, the Bellman equation and the linear solve), Lecture 4 (the optimality test and greedy improvement; the test's uniqueness half is Lecture 5's first consequence of the contraction); Sutton and Barto, Sections 3.5, 3.6, and 4.1.

3 Models and Algorithms in Code [24 points]

`hw1.py`, released with this handout, stores an MDP as the arrays `P[s, a, s2]` $= P(s_2 \mid s, a)$ and `r[s, a]` $= r(s, a)$, a deterministic policy as an integer array, and terminal states as absorbing rows with reward 0; the docstrings are the specification. It provides two ready-made models, the machine of Lectures 3–5 (γ **= 0.8 for the machine reports in this problem**) and the channel of Problem 2; the geometry of the Gridworld below (cells, moves, exits, the start); and two helpers, `induced_chain`, the chain (P^π, r^π) of a policy, and `optimality_test`, Lecture 4's test built from your `policy_evaluation` and `q_from_v`. You write five functions: four algorithms of Lectures 3–5 and the builder of the Gridworld. Only NumPy may be used.



The Gridworld of Lecture 2: eleven cells, four actions; a move goes where intended with probability 0.8 *and slips to each side with probability* 0.1*; a move into the wall or the border stays put. Every move costs the step reward (*−0.04 *unless changed), entering an exit adds* ±1*, and the exits are terminal. Start at* $(1, 1)$*;* $R_{\max} = 1.04$*.* **Use γ = 0.9 in Problems 4(a) and 4(c); Problem 4(b) varies γ.**

(a) **[16 points]** Complete the four algorithm functions marked `TODO`: `policy_evaluation`, the linear solve of Lecture 3 (4 points); `q_from_v`, the lookahead table (3); `greedy_policy`, greedy extraction with the tie rule of its docstring (3); `value_iteration`, Lecture 5's box with its stopping rule (6).

(b) **[8 points]** Complete the model builder marked `TODO`, `gridworld`, which returns `P` and `r` for the Gridworld above at a given step reward.

Run `python hw1.py`: it prints your functions' results on the specified inputs, then runs the experiments of Problem 4 (a block that needs an unwritten function is skipped) and saves the figure. Problem 3 is graded on the reported numerical results. Paste the complete printed output here; Problem 4 is answered from it.

Solution.

```
Python 3.10.12, NumPy 2.2.6
[RESULT] policy_evaluation: machine, gamma = 0.8;
                            never repair: V = [22.0408, 8.5714, 0.0];
                            repair when broken: V = [25.4144, 14.4751, 10.3315]
[RESULT] q_from_v: machine, gamma = 0.8, V = [12.0, -3.0, 2.0];
                   Q(., run) = [13.6, 2.9333, 1.6];
                   Q(., repair) = [-0.4, -0.4, -0.4]
[RESULT] greedy_policy: Q: pi = [1, 0, 1]; Qt (tol = 1e-9): pi = [0, 1, 0]
[RESULT] value_iteration: machine, gamma = 0.8, V_0 = [1.0, 2.0, 3.0];
                          V_1 = [9.067, 5.867, 2.4], V_2 = [14.4, 7.769, 1.92];
                          V_3 = [17.752, 8.655, 1.536];
                          V_K = [25.4144, 14.4751, 10.3315];
                          K = 109 iterations to theta = 1e-10
[RESULT] gridworld: step -0.2, P rows in GW_CELLS order;
                    P((2, 1),E)=[0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.2, 0.8, 0.0], r=-0.20;
                    P((3, 2),N)=[0.0, 0.0, 0.8, 0.0, 0.0, 0.1, 0.1, 0.0, 0.0, 0.0, 0.0], r=-0.30;
                    P((3, 3),S)=[0.0, 0.1, 0.0, 0.1, 0.0, 0.8, 0.0, 0.0, 0.0, 0.0, 0.0], r=-0.10;
                    exit (4, 3): P(s,a,s)=[1.0, 1.0, 1.0, 1.0], r=[0.0, 0.0, 0.0, 0.0];
                    exit (4, 2): P(s,a,s)=[1.0, 1.0, 1.0, 1.0], r=[0.0, 0.0, 0.0, 0.0];
                    step +0.1: rewards for these rows = [0.1, 0.0, 0.2];
                    max |P(+0.1) - P(-0.2)| = 0
4(a) Gridworld, gamma = 0.9, pi_N = 'always N': policy evaluation and the optimality test
    V^(pi_N) by the linear solve (row y = 3 first); V^(pi_N)((1,1)) = -0.3213
    -0.301 -0.190  0.153  0.000
```

```
    -0.313   wall -0.033  0.000
    -0.321 -0.303 -0.175 -0.940
    the test: gap(s) = max_a Q^(pi_N)(s,a) - V^(pi_N)(s), 0.000 where pi_N passes
     0.068  0.226  0.618  0.000
     0.000   wall  0.000  0.000
     0.006  0.082  0.000  0.589
    pi_N fails in 6 of the 9 decision cells; the largest gap, 0.618, is at (3, 3)
4(b) Gridworld value iteration, step -0.04, theta = 1e-08
    K: iterations until the change |V_k - V_{k-1}| is <= theta.   k(pi*): first k whose
    greedy policy is optimal.   ratio: |V_K - V*| / |V_{K-1} - V*|, the factor by which
    the error shrinks in the last iteration.
    gamma    K    k(pi*)  V*((1,1))   ratio
    0.5      20   3       -0.0454     0.456
    0.9      26   3       0.3508      0.438
    0.99     32   8       0.6599      0.453
    gamma = 0.9: V* (row y = 3 first) and the optimal policy
    V*:                              pi*:
    0.581  0.732  0.890  0.000        > > > T
    0.461   wall  0.550  0.000        ^ # ^ T
    0.351  0.300  0.397  0.161        ^ > ^ <
4(c) Gridworld, gamma = 0.9, three step rewards: V*((1,1)) and the optimal policy
    step -0.04: 0.3508        step -2.00: -8.6412       step +0.10: 1.0533
    > > > T                   > > > T                   > > > T
    ^ # ^ T                   ^ # > T                   ^ # < T
    ^ > ^ <                   > > > ^                   ^ < < v
Figure saved: hw1_fig1.png
```

Where this comes from: Lecture 2 (the model as P and r, the Gridworld's transition and reward conventions), Lecture 3 (the chain and the linear solve), Lecture 4 (the lookahead table, greedy extraction with ties, the test), Lecture 5 (value iteration); Sutton and Barto, Sections 4.1 and 4.4. The Gridworld is Russell and Norvig's (Section 17.1).

4 Gridworld [36 points]

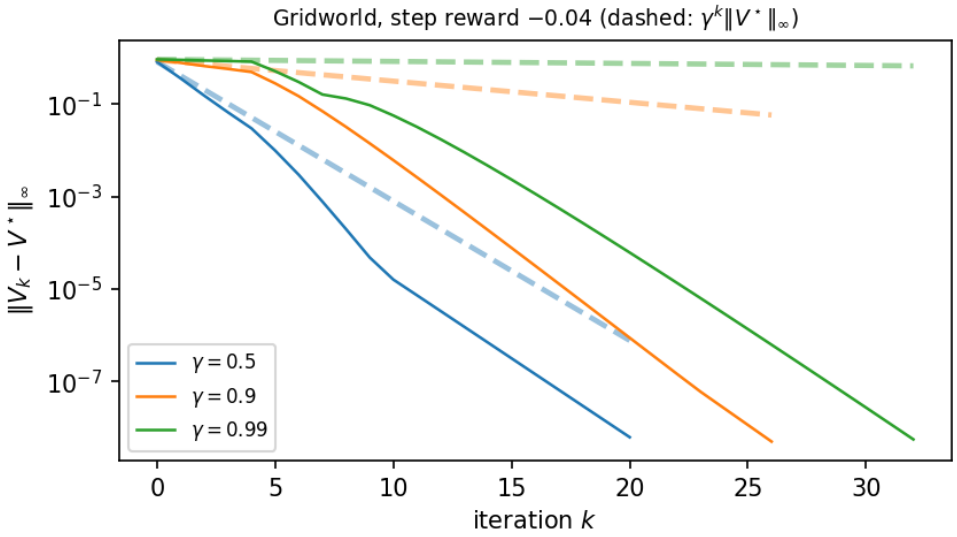
Every entry below is read off the printed output of Problem 3. For a run of value iteration with tolerance θ the output reports K, the number of iterations until the change $\|V_k - V_{k-1}\|_\infty$ is at most θ (Lecture 5's stopping rule; the run returns V_K); $k_{\pi^\star}$, the first k whose greedy policy π_k is optimal, i.e. whose loss $\max_s \left[V^\star(s) - V^{\pi_k}(s)\right]$ is 0; and the ratio $\|V_K - V^\star\|_\infty / \|V_{K-1} - V^\star\|_\infty$, the factor by which the error shrinks in the last iteration.

(a) **[9 points]** The policy π_{N} = "always N" at $\boldsymbol{\gamma = 0.9}$: its exact value at the start and Lecture 4's test. Fill in the table.

$V^{\pi_{\mathrm{N}}}((1,1))$ by the linear solve	-0.3213
decision cells in which π_{N} fails the test	6 of 9
largest gap $\max_a Q^{\pi_{\mathrm{N}}}(s,a) - V^{\pi_{\mathrm{N}}}(s)$, and its cell	0.618 at (3,3)

(b) **[18 points]** Value iteration, step reward -0.04, $\theta = 10^{-8}$, at $\boldsymbol{\gamma \in \{0.5, 0.9, 0.99\}}$. Fill in the table, and include Figure 1 (`hw1_fig1.png`) below it.

γ	K	$k_{\pi^\star}$	$V^\star((1,1))$	ratio
0.5	20	3	-0.0454	0.456
0.9	26	3	0.3508	0.438
0.99	32	8	0.6599	0.453



(c) **[9 points]** Value iteration at $\gamma = 0.9$ with step rewards -0.04, -2, and $+0.1$. Fill in $V^\star((1,1))$ and the optimal action in the two cells next to the -1 exit.

step reward	-0.04	-2	$+0.1$
$V^\star((1,1))$	0.3508	-8.6412	1.0533
$\pi^\star((3,2))$	N	E	W
$\pi^\star((4,1))$	W	N	S

Where this comes from: Lecture 3 (the linear solve), Lecture 4 (the optimality test), Lecture 5 (value iteration, the rate γ^k, the iteration count, the policy read off the iterates), Lecture 2 (the step reward and the reward shift); Sutton and Barto, Sections 4.1 and 4.4.

Assistance Statement (required)

List every person and resource, including AI tools, that you used beyond the course materials, and for what: your collaborators (at most two) by name, any reference you consulted, and any AI tool with the parts it touched. Write "none" if none. You must be able to explain everything you submit.

Solution.

use AI to search numpy functions related to matrix calculation.
Ask AI how to insert image into latex file